# *EyeSeeYou*: A Mobile Assistant for the Visually Impaired

*Rosario Ruisi, Leonardo Cecchini, Matteo Protopapa*
r.ruisi2@studenti.unipi.it , l.cecchini3@studenti.unipi.it , m.protopapa3@studenti.unipi.it

**ABSTRACT**
EyeSeeYou is a mobile application designed to assist visually impaired individuals in safely navigating their environment. The app uses ARCore to analyze the surroundings and detect elements such as steps, slopes, obstacles, and other potential hazards. It provides real-time audio feedback via a text-to-speech system and optional vibration feedback, both configurable by the user. The application is fully functional offline, ensuring accessibility even in areas without internet connection. EyeSeeYou does not yet recognize specific object types but is designed to be extensible and can be improved in the future to increase accuracy and contextual awareness. Step detection is achieved by using the ARCore hitTest() function on points near the bottom of the screen to retrieve their world coordinates and infer elevation changes.

## 1. Introduction

People with visual impairments face daily challenges when walking through unfamiliar or unsafe environments. EyeSeeYou aims to reduce these difficulties by offering an intelligent and lightweight solution that alerts users to obstacles, steps, slopes, or uneven ground through voice and vibration feedback [6]. Unlike complex wearable systems, EyeSeeYou runs entirely on a smartphone and does not require any external connection or server [7][8]. The goal is to increase user independence and safety, while also offering a platform that can be extended in the future with more advanced AI-based object detection [4][5].

## 2. Architecture

EyeSeeYou is designed with a modular structure, where each component plays a specific role integrated within the system. The app leverages ARCore for scene understanding and real-time detection of obstacles, steps, inclines, and declines, supporting the mobility of visually impaired individuals [1].

Communication with the user takes place through two main channels: Text-to-Speech (TTS) and tactile feedback via vibration [2]. EyeSeeYou operates entirely offline: all data processing is performed locally on the device, making the app reliable even in areas with limited or no connectivity [7][8]. User preferences are managed through the Settings Manager, which allows the activation or deactivation of audio and tactile feedback based on personal needs. This component uses Android's SharedPreferences to keep settings persistent across sessions, ensuring a consistent and personalized user experience. The app also supports integration with compatible smartwatches.

### 2.1 HelloARCore

Hello ARCore, is the sample application provided by Google that we started from to create our application. Its structure is organized into two main modules, each with a specific role in managing the AR experience.

At its core is the MainActivity, which acts as the entry point: it handles the initialization of the AR session and the configuration of OpenGL rendering. From here, the ARSession is launched, the heart of the application, which manages the camera, spatial tracking, and the identification of real-world surfaces [1].

The other key component is the Renderer, responsible for drawing the 3D scene. This is where the rendering of detected planes, AR points of interest, and virtual objects that the user can place in the real world takes place. The renderer updates in real-time based on the screen's refresh rate.

The base structure was adapted to optimize performance within our application. Automatic flashlight activation in insufficient ambient lighting conditions was implemented. Unused 3D rendering functionalities were removed. The image rendering frequency was changed from CONTINUOUSLY to WHEN_DIRTY, with an update manually set to 30 Hz.

Concurrently, the camera framerate was reduced to 30 fps.

### 2.2 Obstacle Detection

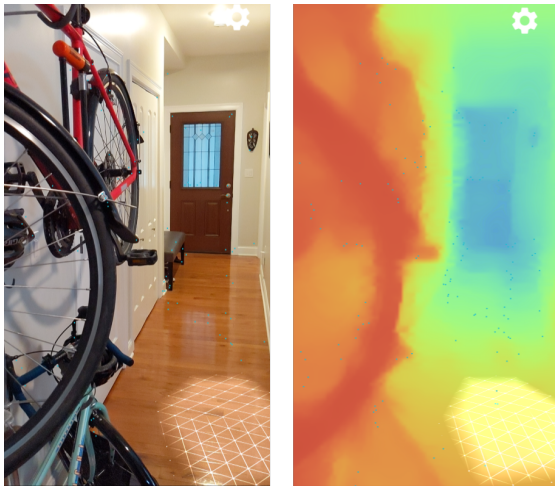

**Figure 1: ARCore Environment Understanding Module**

The obstacle detection module simultaneously employs depth images and semantic images to enable a more specific and accurate analysis of potential obstacles.

Initially, the module synchronously acquires both types of images. Subsequently, it processes a map that associates each semantic label [1] provided by ARCore with all the points corresponding to that object, along with their respective depths, detected within a 3.5-meter radius, defined as our limit of interest for clutter assessment.

In a second phase, for each identified semantic label, the system verifies the presence of points within a 2-meter radius, considered the threshold for defining a "nearby object," saving them in a corresponding map.

This double verification, performed on both the general clutter (3.5m) and the object's proximity (2m), is implemented to understand the actual space occupied by the framed objects and to exclude irrelevant areas, such as flooring.

Each semantic label is then associated with a set of spatial zones that describe its distribution relative to the user and the intended path. The zones considered are: central, high, low, left, right, left wall, and right wall. These zones were determined through a prospective analysis process based on the output of the mobile device's camera when positioned at chest level.

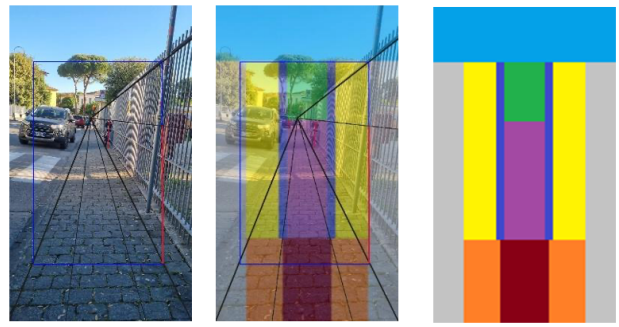

**Figure 2: ARCoreEnvironment Understanding Module**

Finally, sequentially acquired frames are compared using a sliding window approach on groups of three. An obstacle is confirmed in a specific zone if detected in at least 66% of the analyzed frames (i.e., 2 out of 3 times). Based on the outcome of this analysis, a notification message is generated.

### 2.3 Step Detection

To enhance EyeSeeYou's ability to detect terrain changes, a specific step detection feature has been implemented using a 3x4 grid of points positioned in the lower center of the screen. The grid's rows are numbered 1 to 3, where row 1 is farthest from the user and row 3 is the closest. This layout simplifies stair detection, as stairs typically start from ground level. The method relies on converting screen coordinates to real-world coordinates using ARCore's hit test function [1].

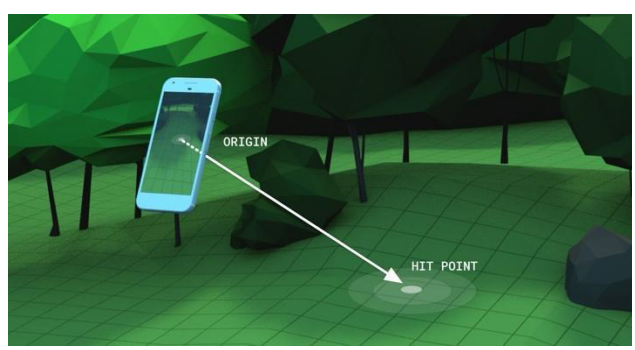


**Figure 3: Hit test working example**

This function returns multiple real-world points from screen coordinates (x, y); the average of these points is used to reduce noise and errors. The app then calculates

the average height and depth differences between the grid rows in three steps:

- ➢ Between row 1 and row 3 (furthest apart), for a quick initial check.
- ➢ Between row 2 and row 3 (closest to the user).
- ➢ Between row 1 and row 2, as a final verification.

These values allow the system to detect three types of ground changes: upward steps, downward steps, or pits. Specifically, a pit is detected when there is a significant difference in both height and depth, and the points are within 120 cm of the camera, to avoid false positives from distant stairs. For steps, the height difference must be between 15 and 40 cm, and the depth difference must not exceed 40 cm; the sign of the height difference indicates upward (positive) or downward (negative) steps.

To ensure timely alerts, the system stores the Pose of detected steps, as the camera might lose sight of them when approaching. The Pose of a new step is compared with the last one detected to determine if it belongs to the same staircase, based on progression in height and depth. If it's a new step or not part of a known staircase, the position is stored and, once the user gets close, an alert is triggered, after which the Pose is removed.

This strategy improves user safety and spatial awareness while navigating environments with elevation changes.

### 2.4 Voice Assistant

EyeSeeYou's voice assistant converts system alerts into spoken messages using the device's built-in Text-to-Speech (TTS) engine [2]. This module supports multiple languages and automatically adapts to the device's current language settings, ensuring accessibility and inclusivity for a wide range of users.

Currently, the app supports English and Italian, with the possibility to expand to additional languages in the future.

Thanks to this feature, the app provides immediate and clear audio feedback, enhancing usability especially for visually impaired users or in situations where visual interaction is limited.

**Figure 4: Voice Assistant**

### 2.5 Voice Command Recognition and Control

EyeSeeYou includes a Speech-to-Text module that allows users to interact with the app using voice commands, eliminating the need to touch the screen and ensuring full accessibility for visually impaired users.

Speech recognition operates in synchronous mode: audio data is sent to Android's built-in API, which processes the entire message and returns a result only after the phrase has been spoken.

A key feature of the app is continuous listening: there's no need to press any button to activate voice recognition. The app always listens in the background, ready to receive commands at any time. This ensures immediate support without disrupting the user's natural interaction with their surroundings.

To further enhance accessibility, voice commands can also be triggered via the touch interface, offering a useful alternative in noisy environments or for users who prefer not to speak.

The process works as follows:

- ➢ The app initializes the Android SpeechRecognizer at startup.

- Spoken commands are compared against predefined keyword lists, organized by category and dynamically loaded from strings.xml.

- Upon recognizing a command, the corresponding action is executed, which may include:

  - Starting or pausing AR viewing mode.
  - Enabling or disabling vibration on the phone or a paired smartwatch via Bluetooth.
  - Turning audio feedback (TTS) on or off.

In case of an error or no input, the system automatically restarts listening, ensuring a continuous and seamless experience. This integration makes EyeSeeYou an always-on, intuitive, and customizable voice assistant, significantly improving the autonomy of blind or visually impaired users.

### 2.6 Vibration Manager

The system includes a component that provides tactile feedback through the device's vibration motor. This function can be toggled on or off based on user preferences, offering a personalized and adaptable experience.

We chose to implement this type of feedback to provide a less intrusive alternative to voice alerts, especially in situations where audio feedback may be disturbing, inappropriate, or hard to hear. This allows the app to discreetly yet effectively adapt to the context and individual user needs.

Additionally, EyeSeeYou supports optional integration with compatible Android smartwatches, enabling the transfer of vibration signals directly to the user's wrist. Once paired, the watch receives the tactile feedback, significantly improving signal perception.

This functionality enhances system flexibility, offering an additional level of accessibility and support in the dynamic contexts of daily life.

## 3. Experimental Results

This section presents the results obtained from experimental tests conducted to evaluate the performance of the EyeSeeYou application in real-world operational scenarios.

### 3.1 Experimental Setup

Tests were carried out on a standard Android device running the application in both indoor and outdoor environments. Experimental conditions included walking near stairs, encountering static and dinamic obstacles such as furniture and walls, and navigating uneven surfaces. The trials were repeated under varying lighting conditions to assess the robustness of the ARCore-based perception system, particularly depth tracking and spatial recognition [1].

### 3.2 Results and Analysis

- **Obstacle and Elevation Detection:** The system demonstrated satisfactory accuracy in detecting obstacles and terrain discontinuities such as steps and stairs. Step detection precision was inversely proportional to the user-to-obstacle distance, with optimal performance within a range of less than 1 meter, while obstacle detection performed better outdoors and in wider areas, having many interferences in tighter zones [1].

- **Impact of Lighting Conditions:** Detection performance was highly dependent on ambient lighting. Under optimal lighting, ARCore provided consistent and accurate depth tracking, while in low-light environments signal degradation resulted in increased localization errors and reduced obstacle recognition frequency [1].

- **Voice Assistant Interaction:** The audio feedback system ensured prompt and clear responses with minimal latency, delivering useful real-time information to the user and enhancing overall system usability [2].

- **Smartwatch-Based Feedback Evaluation:** Additional tests with a paired smartwatch, connected via Bluetooth, confirmed the effectiveness of external vibration transmission. Users reported improved perception of feedback

when the smartwatch was worn on the wrist and the phone was not directly accessible, highlighting the usefulness of wearable integration through Bluetooth connectivity [7][8].

- **Technical Limitations:** The current application had to work on low refresh rate due to the complexity of the operations performed by the application, this comports a reduction on the precision of the semantic image and depth image [1].

### 3.3 Future Work

Experimental results confirm that EyeSeeYou can reliably detect environmental obstacles and generate timely audio feedback, with optimal performance observed in well-lit conditions and during outdoor use [1][2]. Future developments should focus on optimizing application operations and reducing resource consumption. Furthermore, planned enhancements include dynamically detecting the type of environment to adjust app settings accordingly, and dynamically adapting the message rate to the user's walking speed.

## 4. Conclusion

This work presented *EyeSeeYou*, a mobile application developed to assist visually impaired users in safely navigating their surroundings by leveraging ARCore-based spatial analysis [1] and providing real-time feedback. The system effectively detects generic obstacles and elevation changes, delivering audio [2] and haptic alerts to enhance users' situational awareness.

Experimental evaluation confirmed that the application performs reliably under well-lit conditions and when users are in close proximity to environmental hazards. The integrated voice assistant demonstrated efficacy in providing timely and comprehensible feedback [2]. Nonetheless, challenges persist in low-light scenarios and in the current system's limited capability to semantically classify specific obstacle types.

Future developments will aim to enhance obstacle recognition through the integration of machine learning-based object detection models such as YOLO [4] or MediaPipe [5], enabling the application to differentiate between various object categories. Additional planned improvements include the incorporation of GPS-enabled navigation, wearable device compatibility, and increased robustness across diverse lighting environments. *EyeSeeYou* represents a significant advancement toward accessible, real-time environmental awareness for visually impaired individuals, with strong potential for ongoing refinement and practical deployment.

## REFERENCES


[1] Google. "ARCore Overview," [Online]. Available: https://developers.google.com/ar/discover

[2] Android Developers. "TextToSpeech | Android Developers," [Online]. Available: https://developer.android.com/reference/android/speech/tts/TextToSpeech

[3] Android Developers. "SpeechRecognizer | Android Developers," [Online].
Available: https://developer.android.com/reference/android/speech/SpeechRecognizer

[4] Redmon, J., Farhadi, A. "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[5] Lugaresi, C., et al. "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.

[6] World Health Organization. "World report on vision," 2019.

[7]Object Detection in the Context of Mobile Augmented Reality: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9284738

[8]MobiEye: An Efficient Shopping-Assistance System for the Visually Impaired With Mobile Phone Sensing: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10255121